

Operating Systems: Concurrency and Synchronization (CS 301)

Lecture 7: Race Conditions and Critical Sections

1. Introduction to Concurrency

Concurrency is the execution of multiple instruction sequences at the same time. In modern operating systems, c

2. Critical Section and Race Conditions

A race condition is an undesirable situation that occurs when a device or system attempts to perform two or more

A critical section is a piece of code that accesses shared resources (data or hardware device) that must not be c

To guarantee correct execution, a solution to the critical section problem must satisfy three requirements:

1. Mutual Exclusion: If process P_i is executing in its critical section, then no other processes can be executing in
2. Progress: If no process is executing in its critical section and there exist some processes that wish to enter the
3. Bounded Waiting: A bound must exist on the number of times that other processes are allowed to enter their c

Operating Systems: Concurrency and Synchronization (CS 301)

Lecture 8: Mutexes, Semaphores, and Condition Variables

1. Mutex Locks (Mutual Exclusion)

A mutex is a locking mechanism used to synchronize access to a resource. Only one task can acquire the mutex.

2. Semaphores

A semaphore is a synchronization tool introduced by Edsger Dijkstra. A semaphore S is an integer variable that,

- $\text{wait}(S)$: Decrements the semaphore value. If the value becomes negative, the calling process is blocked.
- $\text{signal}(S)$: Increments the semaphore value. If there are blocked processes, one is unblocked.

Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.

Binary semaphores behave similarly to mutex locks with value either 0 or 1, but without ownership restrictions.

3. Classic Synchronization Problems

Classic problems include:

- The Producer-Consumer Problem (Bounded-Buffer Problem)
- The Readers-Writers Problem
- The Dining Philosophers Problem

Operating Systems: Concurrency and Synchronization (CS 301)

Lecture 9: Deadlocks and Coffman Conditions

1. Definition of Deadlock

In a multiprogramming environment, several processes may compete for a finite number of resources. A process

2. The Four Coffman Deadlock Conditions

A deadlock situation can arise if and only if the following four conditions hold simultaneously in a system:

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode.
2. Hold and Wait: A process must be concurrently holding at least one resource and requesting additional resources.
3. No Preemption: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it.
4. Circular Wait: A closed chain of processes exists, such that each process holds at least one resource needed by another process in the chain.

3. Deadlock Handling Strategies

Modern operating systems use three primary strategies:

- Deadlock Prevention or Avoidance (Banker's Algorithm)
- Deadlock Detection and Recovery
- Ignore the problem entirely (the Ostrich Algorithm, commonly used in Unix and Windows)